



DOCUMENTAZIONE SMARTPRICE HUNTER

Progetto Programmazione Web e Mobile a.a. 2025/2026



CIACCIO DAVIDE
MANDRACCHIA GIUSEPPE
PACE RICCARDO

Sommario

1. Descrizione dell'applicazione	3
1.1 Target di prodotti	3
2. Funzionalità	3
2.1 Pagina di Login	3
2.1.1 Variabili di stato e Attributi	4
2.1.2 Funzionalità e flusso logico	4
2.1.3 API esterne e dettagli tecnici	5
2.1.4 Mockup Pagina di Login	5
2.2 Pagina di Registrazione	6
2.2.1 Variabili di stato e Attributi	6
2.2.2 Funzionalità e flusso logico	6
2.2.3 Mockup Pagina di Registrazione	8
2.3 Pagina di Dashboard	9
2.3.1 Variabili di stato e Attributi	9
2.3.2 Funzionalità e flusso logico	9
2.3.3 Mockup Pagina di Dashboard	12
2.4 Pagina di Storico prezzi	13
2.4.1 Variabili di stato e Attributi	13
2.4.2 Funzionalità e flusso logico	13
2.4.3 Mockup Pagina di Storico prezzi	15
2.5 Pagina di Fotocamera	17
2.5.1 Variabili di stato e Attributi	17
2.5.2 Funzionalità e flusso logico	17
2.5.3 API esterne e dettagli tecnici	19
2.5.4 Mockup Pagina di Fotocamera	20
2.6 Pagina di Comparazione prezzi	21
2.6.1 Variabili di stato e Attributi	21
2.6.2 Funzionalità e flusso logico	21
2.6.3 API esterne e dettagli tecnici	23
2.6.4 Mockup Pagina di Comparazione prezzi	23

2.7 Pagina di Mappa	24
2.7.1 Variabili di stato e Attributi.....	24
2.7.2 Funzionalità e flusso logico	24
2.7.3 API esterne e dettagli tecnici	26
2.7.4 Mockup Pagina di Mappa	26
2.8 Pagina di Avvisi	27
2.8.1 Variabili di stato e Attributi.....	27
2.8.2 Funzionalità e flusso logico	27
2.8.3 Mockup Pagina di Avvisi	29
2.9 Pagina di Gestione account	30
2.9.1 Variabili di stato e Attributi.....	30
2.9.2 Funzionalità e flusso logico	30
2.9.3 Mockup Pagina di Gestione account	33
2.10 Pannello Admin (raggiungibile solo attraverso l'url http://localhost:8100/admin-users)	34
2.10.1 Variabili di stato e Attributi.....	34
2.10.2 Funzionalità e flusso logico	34
2.10.3 Mockup Pannello Admin	36



SmartPrice Hunter

1. Descrizione dell'applicazione

SmartPrice Hunter è un'applicazione web progettata per guidare gli utenti verso uno shopping intelligente e consapevole. Le sue due funzionalità principali, il **monitoraggio** e il **confronto dei prezzi**, permettono di individuare sempre l'offerta migliore.

L'applicazione si distingue inoltre per un'innovativa funzionalità di mappatura dinamica dei prodotti. Sfruttando il **crowdmapping***, connette direttamente i consumatori attraverso una mappa costantemente aggiornata in tempo reale, alimentata dal contributo collettivo e simultaneo di tutta la community.

crowdmapping* : Il crowdmapping (da *crowd*, folla, e *mapping*, mappatura) è una forma di cartografia collaborativa e partecipativa in cui le mappe digitali vengono generate, aggiornate e arricchite dagli utenti stessi. Sfruttando la geolocalizzazione, raccoglie dati condivisi dal basso attraverso il web o le app mobili.

1.1 Target di prodotti

Considerando l'elevata volatilità che caratterizza il mercato tecnologico odierno, SmartPrice Hunter si concentra specificamente sul settore dell'**elettronica**.

La funzionalità di monitoraggio dei prezzi, accessibile dalla pagina /dashboard, è progettata e ottimizzata specificamente per elaborare gli URL dei prodotti di amazon.it appartenenti alla categoria "Elettronica".

Si noti che con questo, non si esclude che lo script di scraping possa andare a segno su altre categorie di prodotti Amazon o su altre piattaforme di e-commerce.

2. Funzionalità

2.1 Pagina di Login

La pagina di Login è il portale di accesso all'applicazione. La pagina presenta due form in cui l'utente registrato può inserire le proprie credenziali (indirizzo e-mail

e password). Inoltre, la schermata presenta un bottone "Crea Account" che se cliccato indirizza l'utente non registrato verso la pagina di Registrazione.

2.1.1 Variabili di stato e Attributi

Le principali variabili di stato e gli attributi definiti all'interno della classe LoginPage includono:

- credentials: L'oggetto delegato a memorizzare e-mail e password inseriti dall'utente.

2.1.2 Funzionalità e flusso logico

Autenticazione dell'Utente e Inizializzazione della Sessione

- Funzione/i di riferimento: onLogin() (nel componente login.page.ts), login() (nel servizio auth.ts).
- Logica e Flusso Dati: Al click del bottone "Accedi", il metodo onLogin() recupera lo stato attuale dell'oggetto credentials e lo passa al servizio di autenticazione.

Tale servizio prepara una richiesta asincrona verso (/api/auth/login), l'infrastruttura backend. Qui entra in gioco il controller di login dentro il file authController.js.

La risposta del server viene analizzata nel service attraverso la funzione tap(). Se il server conferma la validità delle credenziali il servizio esegue un'operazione per consolidare la sessione: salva in modo persistente nella localStorage (memoria del browser) il token di sicurezza, l'identificativo univoco dell'utente e un UserObject contenente username e role dell'utente (user o admin), essenziali per verificare i privilegi amministrativi dell'utente loggato.

Infine all'interno della funzione di onLogin() nel componente , Se la risposta dal backend è positiva, l'utente viene instradato verso la dashboard principale; altrimenti il sistema genera un alert visivo "Credenziali non valide, riprova!".

Transizione verso il Modulo di Registrazione

- Funzione/i di riferimento: goToRegister() (nel componente login.page.ts).
- Logica e Flusso Dati: Intercettando l'intenzione dell'utente, questo metodo invoca il gestore di navigazione interno (router) per navigare alla pagina di registrazione (/register).

2.1.3 API esterne e dettagli tecnici

Meccanismo di Autenticazione Stateless tramite JSON Web Token (JWT)

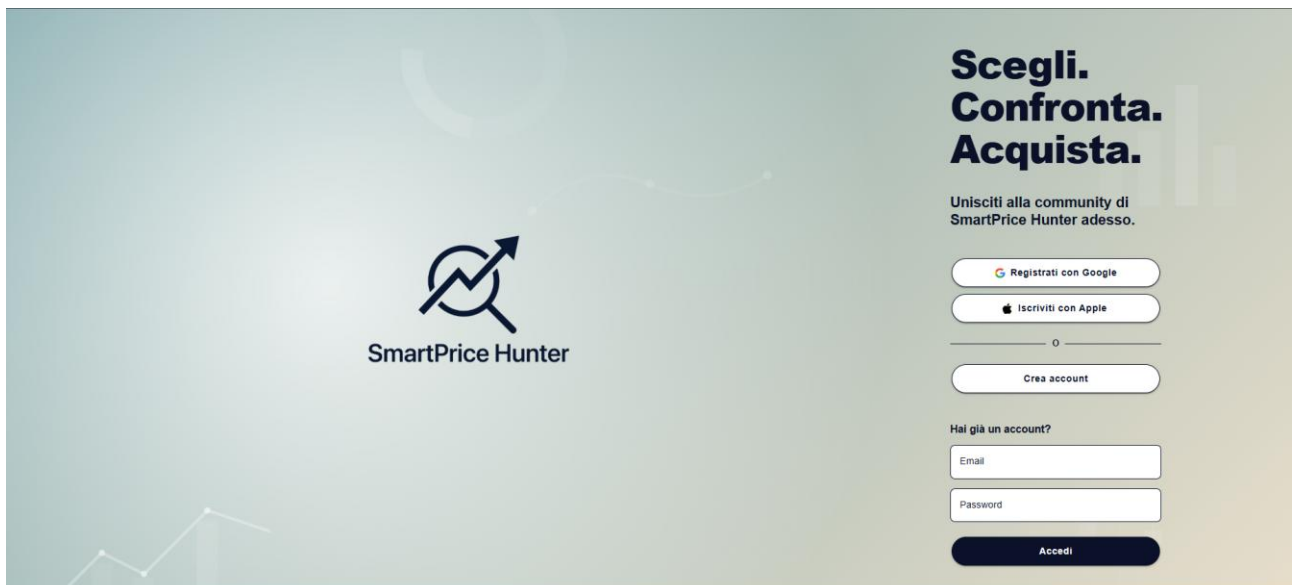
L'architettura di "SmartPrice Hunter" adotta un modello di autenticazione basato su JSON Web Token (JWT).

A seguito del successo della chiamata API verso l'endpoint di login, il backend genera una stringa crittografica, const token, attraverso la funzione `sign()` appartenente al modulo esterno `jsonwebtoken`. Questa stringa racchiude in forma cifrata le informazioni non sensibili dell'utente (`userId` e `role`) "firmate" da una chiave `JWT_SECRET` all'interno del file `.env`.

Nelle successive interazioni con l'applicativo, la presenza del token memorizzato all'interno degli header delle richieste HTTP (tipicamente sotto la chiave *Authorization: Bearer*), consentono al backend di verificare istantaneamente l'identità e i permessi dell'utente senza dover interrogare nuovamente il database.

È previsto che il token scada dopo 24h.

2.1.4 Mockup Pagina di Login



Scegli. Confronta. Acquista.

Unisciti alla community di SmartPrice Hunter adesso.

[Registrati con Google](#)

[Iscriviti con Apple](#)

[Crea account](#)

Hai già un account?

[Accedi](#)

2.2 Pagina di Registrazione

La pagina di registrazione offre ai nuovi utenti un'iscrizione veloce.

2.2.1 Variabili di stato e Attributi

Le principali variabili di stato e gli attributi definiti all'interno della classe RegisterPage includono:

- **UserData:** L'oggetto delegato a memorizzare i campi inseriti dall'utente nel modulo di registrazione (username, email, password, confirmPassword).
- **showPassword e showConfirmPassword:** Flag booleani delegati al controllo della visibilità dei campi password.
- **errors:** L'oggetto delegato per mappare e isolare i messaggi di errore specifici per ciascun campo di input.
- **passwordStrength:** Variabile intera che quantifica il livello di sicurezza della password (da 0 a 3) in fase di digitazione.
- **forbiddenUsernames e commonPasswords:** Array costanti utilizzati come *blacklist* sul frontend per inibire scelte poco sicure o riservate.

2.2.2 Funzionalità e flusso logico

Miglioramento dell'Esperienza utente e visibilità

- **Funzione/i di riferimento:** `togglePassword()`, `toggleConfirmPassword()`.
- **Logica e Flusso Dati:** Queste due funzioni sono attivate dall'interazione dell'utente con delle apposite icone a forma d'occhio nei due form delegati all'inserimento della password nel modulo di registrazione, queste funzioni si limitano a invertire lo stato logico (da vero a falso o viceversa) dei flag di visibilità. Il template reagisce a questo cambio di stato modificando l'attributo HTML dell'input da "password" a "text".

Sanitizzazione e valutazione dell'Input dell'utente

- **Funzione/i di riferimento:** `formatUsername()`, `onPasswordInput()`.
- **Logica e Flusso Dati:** Il metodo `formatUsername()` elimina gli spazi bianchi laterali (`.trim()`) e forza i caratteri in minuscolo (`.toLowerCase()`). Il metodo `onPasswordInput()` viene invece innescato ad ogni singola digitazione dell'utente nel campo password. Esso valuta progressivamente la complessità della stringa allocando punti (incrementando la variabile `passwordStrength`) in base al soddisfacimento di criteri algoritmici specifici, verificati tramite espressioni regolari (presenza di maiuscole, numeri, caratteri speciali e lunghezza complessiva).

Validazione del modulo di registrazione

- **Funzione/i di riferimento:** `validateForm()`.

- Logica e Flusso Dati: Prima di autorizzare la sottomissione dei dati, questa funzione agisce per ogni campo del modulo di registrazione.

Azzerare l'oggetto errors.

Successivamente, sottopone l'email, lo username e la password a validazioni strutturali tramite pattern di espressioni regolari (Regex), assicurandosi che rispettino formati standard e non contengano caratteri illeciti.

Il sistema confronta inoltre gli input con le *blacklist* locali (forbiddenUsernames e commonPasswords), bloccando termini riservati.

Infine, verifica la congruenza esatta tra la password e la sua conferma.

La funzione restituisce un valore booleano: falso se riscontra anomalie (valorizzando contestualmente le variabili di errore), vero se a correttezza dei dati inseriti è confermata.

Gestione sottomissione dei dati e interazione con il Backend

- Funzione/i di riferimento: onRegister() (nel componente login.page.ts), register() (nel servizio auth.ts).
- Logica e Flusso Dati: All'attivazione del pulsante "Registrati", la funzione richiede innanzitutto l'autorizzazione di validateForm().

In caso di semaforo verde, il flusso passa al servizio iniettato.

Il metodo register() dell'AuthService prende in carico l'oggetto userData e delega al client HTTP interno la costruzione di una richiesta POST asincrona indirizzata all'endpoint /api/auth/register nel backend.

Qui entra in gioco il controller di Registrazione nel file authController.js.

- In caso di risposta positiva dal server, il sistema genera un alert di conferma e invoca il gestore di navigazione per instradare l'utente alla pagina di Login.
- In caso di errore dal server, il sistema genera un alert di errore.

Transizione verso l'Accesso

- Funzione/i di riferimento: goToLogin().
- Logica e Flusso Dati: Qualora l'utente scelga di annullare l'operazione, questo metodo invoca il gestore di navigazione interno (router) per navigare alla pagina di login (/login).

2.2.3 Mockup Pagina di Registrazione

Crea account

Unisciti a noi!



SmartPrice Hunter

Username

Email

Password

Conferma Password

Registrati

Hai già un account? [TORNA AL LOGIN](#)

© 2026 SmartPrice Hunter

2.3 Pagina di Dashboard

La Dashboard consente di utilizzare la funzionalità di **monitoraggio dei prezzi online**. La pagina si articola in due sezioni principali:

Le cards superiori: presentano tre cards, di cui due contengono i dati riassuntivi relativi ai prodotti monitorati, mentre la terza contiene un form per l'inserimento dell'URL del nuovo prodotto da tracciare.

La tabella riassuntiva: mostra l'elenco completo di tutti i prodotti in osservazione, riportando informazioni dettagliate quali il prezzo corrente e la relativa variazione di prezzo.

2.3.1 Variabili di stato e Attributi

Le principali variabili di stato e gli attributi definiti all'interno della classe `DashboardPage` includono:

- `products`: Array dinamico che contiene i dati dei prodotti visualizzati nella tabella riassuntiva (subisce mutazioni in base a filtri e ordinamenti).
- `originalProducts`: Array che conserva una copia immutabile dei prodotti nell'ordine originale.
- `newProductUrl`: Stringa delegata a memorizzare l'input testuale per l'inserimento di un nuovo link da analizzare.
- `searchQuery`: Stringa delegata a memorizzare l'input testuale per la barra di ricerca nella tabella riassuntiva.
- `totalProducts`, `changesToday`, `changesWeek`: counter per popolare le due cards statistiche.
- `isLoading`, `isSidebarActive`: Flag booleani per gestire rispettivamente i buffering di caricamento e la visibilità del menu laterale nel mobile.

2.3.2 Funzionalità e flusso logico

Inizializzazione e sincronizzazione dei dati

- Funzione/i di riferimento: `ngOnInit()`, `loadProducts()` (nel componente `dashboard.page.ts`), `getUserProducts()` (nel servizio `product.ts`).
- Logica e Flusso Dati: `ngOnInit()` invoca `loadProducts()` che delega al servizio `ProductService` la richiesta HTTP GET verso il backend per scaricare i prodotti dell'utente. Alla ricezione della risposta positiva dal server, il payload viene copiato in due array distinti:

products (destinato al rendering visivo) e originalProducts (conservato come fonte di verità immutabile). Dopo, il sistema invoca il metodo di calcolo delle statistiche(`calculateStats()`).

Motore di ricerca client-side

- Funzione/i di riferimento: `filterProducts()`, `clearSearch()` (nel componente `dashboard.page.ts`).
- Logica e Flusso Dati: L'applicazione implementa un motore di ricerca operante esclusivamente sul frontend, senza interrogare il server. All'immissione di ogni singolo carattere nel campo di testo, la funzione normalizza la stringa rimuovendo gli spazi laterali (`.trim()`) e forzando i caratteri in minuscolo (`.toLowerCase()`). L'algoritmo filtra l'array `originalProducts`, verificando l'inclusione della sottostringa all'interno del nome del prodotto, e proietta i soli risultati positivi nell'array visibile `products`. Qualora la barra di ricerca venga svuotata, o venga interpellato il metodo `clearSearch()`, il sistema ripristina la visualizzazione originaria.

Avvio motore di scraping e aggiunta prodotto

- Funzione/i di riferimento: `startScraping()`.
- Logica e Flusso Dati: Questo metodo è attivato dal click sul bottone "Aggiungi prodotto".

Successivamente, imposta il flag `isLoading=true`, che inibirà un'altra azione sul bottone "Aggiungi prodotto" mostrando un buffering.

La funzione delega al servizio `ProductService` l'invio del link al backend.

Il `ProductServices` invia una richiesta HTTP POST verso `/api/products/add` al backend.

Qui entra in gioco la funzione `addProduct()` dentro il file `scraperController.js` che chiama la funzione `runScrapingEngine()` anche essa dentro lo stesso controller.

Alla ricezione di una risposta di successo (che implica che il server Node.js ha completato lo scraping della pagina esterna e salvato i dati).

Il flag `isLoading` è impostato a `false`, il campo di testo svuotato, l'utente notificato tramite un messaggio di testo "Prodotto aggiunto con successo", viene invocato il metodo `loadProducts()` per aggiornare la tabella.

Rimozione prodotto e aggiornamento tabella

- Funzione/i di riferimento: `deleteProduct()`.

- Logica e Flusso Dati: La funzione `deleteProduct()` prima mostra un messaggio di conferma ("Sei sicuro di voler smettere di monitorare questo prodotto?")

Se "Okay" invia una richiesta DELETE asincrona al server.

Qui, anziché richiedere nuovamente tutta la mole di dati dal server, la funzione elimina l'oggetto corrispondente dagli array locali (`products` e `originalProducts`) e ricalcola le statistiche chiamando la funzione `calculateStatus()`.

Algoritmo di valutazione della variazione di prezzo

- Funzione/i di riferimento: `getVariationData()`.
- Logica e Flusso Dati: Invocata per ogni singola riga della tabella, questa funzione analizza l'array storico dei prezzi del prodotto. Se lo storico contiene almeno due campionamenti, estrapola l'ultimo e il penultimo valore cronologico. Calcola la differenza assoluta e la converte in percentuale, operando un arrotondamento matematico al primo decimale. Ritorna quindi un oggetto strutturato alla vista HTML che decodifica lo stato: "invariato" (se la differenza è prossima allo zero), "aumentato" o "diminuito", determinando la conseguente colorazione semantica (verde/rosso/grigio) nell'interfaccia.

Ordinamento dinamico della griglia

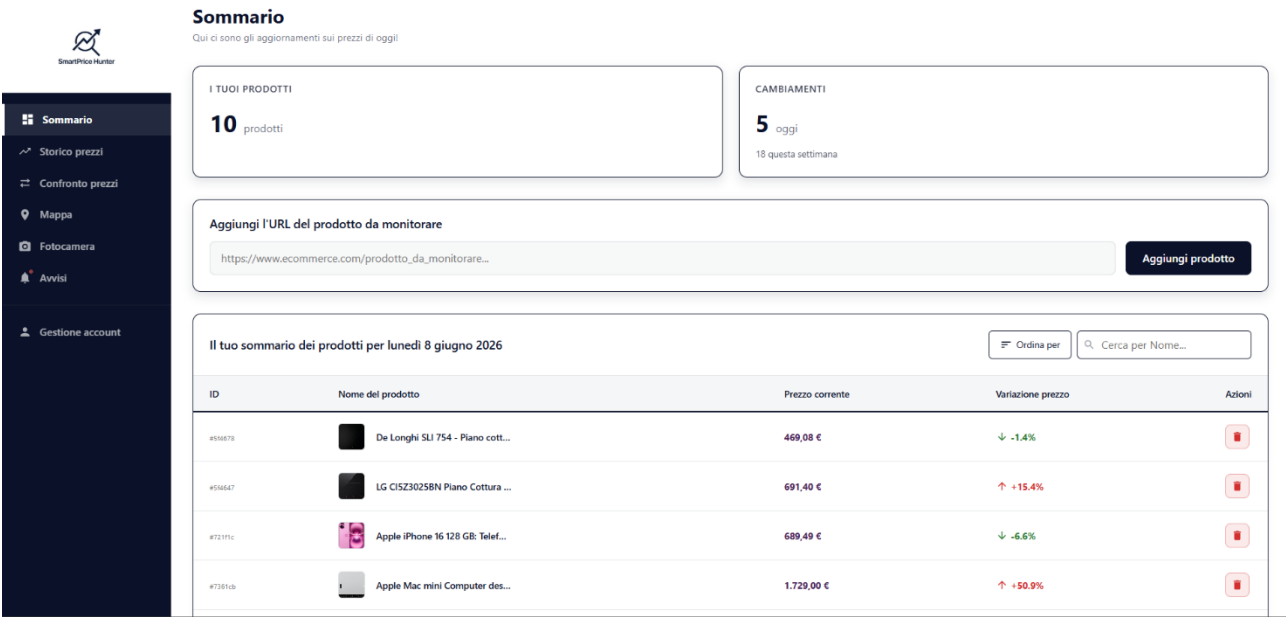
- Funzione/i di riferimento: `presentSortOptions()`, `sortProducts()`.
- Logica e Flusso Dati: Tramite l'infrastruttura di Ionic, la pressione sul bottone di ordinamento genera un Action Sheet nativo (menu a tendina). Alla selezione di un criterio (Crescente, Decrescente o Default), il motore di sorting riorganizza in loco gli elementi dell'array `products`. L'algoritmo di comparazione prevede una logica di tolleranza ai guasti: qualora un prodotto presenti un prezzo nullo o non disponibile ("N/D"), questo viene equiparato a zero per evitare il fallimento del ciclo di ordinamento.

Calcolo matematico delle metriche temporali

- Funzione/i di riferimento: `calculateStats()`.
- Logica e Flusso Dati: Genera i dati statistici delle due cards superiori. Attraversa l'intero array `OriginalProducts`, e per ogni prodotto, ispeziona il relativo storico prezzi. Ignorando il prezzo di base (creazione), confronta i timestamp delle registrazioni successive con due finestre temporali specifiche (l'inizio della giornata odierna e una settimana esatta prima del momento attuale). Accumula in contatori separati le variazioni di prezzo

avvenute nei rispettivi archi di tempo, fornendo all'utente un polso immediato sulla volatilità del mercato monitorato.

2.3.3 Mockup Pagina di Dashboard



2.4 Pagina di Storico prezzi

La pagina dello Storico Prezzi è formata da una tabella in cui a ogni prodotto monitorato viene affiancato un mini grafico integrato direttamente nella riga della tabella, consentendo un colpo d'occhio immediato sulla volatilità del prezzo del prodotto. È prevista la possibilità di cliccare su una singola entità per far emergere una finestra modale (finestra che si sovrappone alla pagina principale) contenente un grafico esplorabile. Completa l'interfaccia una barra di ricerca testuale.

2.4.1 Variabili di stato e Attributi

Le principali variabili di stato e gli attributi definiti all'interno della classe `PriceHistoryPage` includono:

- `products`: Array dinamico che contiene i dati dei prodotti visualizzati nella tabella riassuntiva (subisce mutazioni in base a filtri e ordinamenti).
- `originalProducts`: Array che conserva una copia immutabile dei prodotti nell'ordine originale.
- `charts`: Un dizionario di tipo chiave-valore volto a tracciare le istanze attive dei mini grafici, associando l'ID del prodotto all'oggetto grafico di `Chart.js`.
- `isModalOpen` e `selectedProduct`: Variabili di stato delegate all'attivazione visiva della finestra modale e l'iniezione dei dati del prodotto selezionato al suo interno.
- `modalChart`: Riferimento in memoria all'istanza univoca del grafico dettagliato generato all'interno della modale.

2.4.2 Funzionalità e flusso logico

Inizializzazione e gestione reattiva dello stato

- Funzione/i di riferimento: `ngOnInit()`, `loadProducts()` (nel componente `price-history.page.ts`), `getUserProducts()` (nel servizio `product.ts`).
- Logica e Flusso Dati: `ngOnInit()` invoca `loadProducts()` che delega al servizio `ProductService` la richiesta HTTP GET verso il backend per scaricare lo storico dei prodotti. Il servizio elabora la richiesta includendo il token di autorizzazione negli header. Alla ricezione della risposta, i dati vengono clonati negli array locali `products` e `originalProducts`. A questo punto, l'invocazione della funzione `renderMiniCharts()` per il disegno dei mini grafici viene differita asincronamente tramite un timer di 200 millisecondi, tale funzione fa parte della libreria grafica `Chart.js`.

Motore di ricerca e re-inizializzazione grafica

- Funzione/i di riferimento: `filterProducts()`, `clearSearch()` (nel componente `price-history.page.ts`).
- Logica e Flusso Dati: La stringa di input dell'utente nella barra di ricerca viene normalizzata rimuovendo gli spazi laterali (`.trim()`) e forzata in minuscolo (`.toLowerCase()`). L'algoritmo filtra l'array `originalProducts` verificando l'inclusione della sottostringa all'interno del nome del prodotto, e sovrascrive l'array di visualizzazione `products` proiettando i soli risultati positivi. Qualora la ricerca venga annullata tramite `clearSearch()`, il sistema ripristina la visualizzazione originaria svuotando la query.

Poiché la mutazione dell'array comporta aggiornamenti strutturali nei nodi HTML, entrambe le funzioni pianificano un differimento asincrono di 200 millisecondi per lanciare nuovamente il motore di rendering `renderMiniCharts()`.

Generazione dinamica dei mini grafici

- Funzione/i di riferimento: `renderMiniCharts()` (nel componente `price-history.page.ts`).
- Logica e Flusso Dati: Questa funzione esegue un'iterazione sul vettore dei prodotti attualmente visibili. Questa funzione esegue un'iterazione sull'array `products` attualmente visibile. Per ciascun elemento provvisto di uno storico prezzi, calcola dinamicamente l'identificatore del nodo canvas associato e tenta di recuperarlo dal DOM. Qualora esista già un'istanza grafica precedente per quel prodotto nel dizionario locale `charts`, l'algoritmo invoca esplicitamente il metodo `.destroy()`.

Successivamente, mappa gli array interni dello storico per costruire le ascisse con le date localizzate e le ordinate con i relativi prezzi. In seguito, istanzia un nuovo oggetto grafico minimale, privato di assi e legende per adattarsi perfettamente agli spazi ristretti dell'interfaccia utente.

Colorazione dei segmenti nei grafici

- Funzione/i di riferimento: `segmentColorConfig` (nel componente `price-history.page.ts`).
- Logica e Flusso Dati: Rappresenta una funzione passata direttamente al motore grafico `Chart.js`. Durante il disegno della linea, l'algoritmo interPELLa questa logica per ogni singolo segmento che congiunge due punti contigui. Estrahendo e confrontando il valore sull'asse Y del punto precedente con quello attuale, la funzione restituisce una stringa cromatica differente.

Ritorna il rosso se si verifica un incremento aritmetico del prezzo, il verde se si rileva una diminuzione del prezzo.

Funzione per aprire i grafici in una finestra modale

- Funzione/i di riferimento: `openChartModal()`, `closeModal()` (nel componente `price-history.page.ts`).
- Logica e Flusso Dati: La selezione di uno specifico prodotto tramite `openChartModal()` altera il flag di visibilità `isModalOpen` impostandolo a vero e memorizza il payload nell'oggetto `selectedProduct`. Interviene nuovamente un ritardo asincrono di 300 millisecondi. Viene distrutto l'eventuale grafico preesistente salvato nella variabile `modalChart` e ne viene inizializzato uno nuovo sfruttando una configurazione estesa. Questa nuova configurazione attiva gli assi cartesiani (X e Y), le griglie di riferimento per un'agevole lettura dei valori e abilita *tooltip* interattivi, formattati dinamicamente per mostrare il prezzo in valuta. La funzione `closeModal()` agisce infine in modo speculare: ripristina lo stato iniziale, chiude la modale, azzerava il prodotto selezionato ed elimina l'oggetto grafico dalla memoria invocando `.destroy()`.

2.4.3 Mockup Pagina di Storico prezzi





Storico prezzi

Qui puoi visualizzare lo storico prezzi di ogni tuo prodotto!

Sommario

Storico prezzi

Confronto prezzi

Mappa

Fotocamera

Avvisi

Gestione account

Seleziona un prodotto

Cerca per Nome...

Nome del prodotto

De'Longhi SLI 754 - Piano cottura da 75 cm
Nero

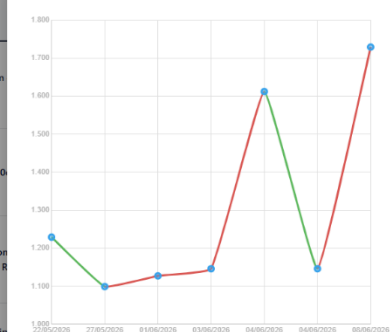
LG C15Z3025BN Piano Cottura Induzione 80
cm

Apple iPhone 16 128 GB: Telefono 5G con Caricamento
autonomia in più. Compatibile con AirPods; Ricarica

Apple Mac mini Computer desktop con chip M4
24 GB di memoria unificata, 512 GB di archiviazione
Compatibile con iPhone/iPad

Apple iPad Air 13" (M4): display Liquid Retina, 128 GB, fotocamera frontale e
posteriore da 12MP, Wi-Fi 7 con chip Apple N1 - Viola

Grafico andamento prezzi per : Apple Mac mini Computer d... CHIUDI



2.5 Pagina di Fotocamera

La pagina della Fotocamera funge da strumento principale per l'inserimento dei prodotti nell'applicazione. Attraverso un percorso guidato (*wizard*), l'utente può inquadrare un codice a barre, ottenere i dettagli del prodotto, inserire il prezzo rilevato fisicamente e associarlo a un punto vendita geolocalizzato nelle proprie vicinanze.

2.5.1 Variabili di stato e Attributi

Le principali variabili di stato e gli attributi definiti all'interno della classe `CameraPage` includono:

- `step`: Variabile stringa che gestisce la macchina a stati del wizard, governando la visibilità delle interfacce (valori possibili: `scan`, `found`, `price`, `store`).
- `scannedProduct`: Oggetto delegato a memorizzare e consolidare progressivamente i dati del prodotto durante i vari passaggi dell'inserimento (barcode, nome, marca, immagine, prezzo, negozio).
- `manualBarcode`: Stringa delegata a memorizzare l'input testuale dell'utente in caso di inserimento manuale del codice, usato come meccanismo di fallback.
- `nearbyStores` e `filteredStores`: Array dinamici adibiti a memorizzare e manipolare la lista degli esercizi commerciali rilevati nelle vicinanze dell'utente.
- `isLoadingStores`: Flag booleano per governare il feedback visivo (*buffering*) durante le richieste di geolocalizzazione e mappatura di rete.

2.5.2 Funzionalità e flusso logico

Acquisizione del codice a barre

- Funzione/i di riferimento: `startScan()`, `submitManualBarcode()` (nel componente `camera.page.ts`).
- Logica e Flusso Dati: Il metodo `startScan()` attiva l'interfaccia hardware nativa del dispositivo per la lettura ottica tramite il plugin Capacitor. Alla rilevazione di un codice a barre, estrae la stringa alfanumerica risultante. Parallelamente, il metodo `submitManualBarcode()` intercetta l'inserimento testuale puro. In entrambi i rami di esecuzione, la stringa acquisita viene iniettata alla funzione `lookupProduct()` per avviare il processo di risoluzione.

Ricerca e risoluzione del prodotto

- Funzione/i di riferimento: `lookupProduct()` (nel componente `camera.page.ts`), `lookupBarcode()` (nel controller `scanController.js`).
- Logica e Flusso Dati: L'applicazione costruisce una richiesta HTTP GET verso il backend trasmettendo il codice a barre letto. Il controller di destinazione orchestra una ricerca a due livelli. Dapprima interroga la collezione MongoDB locale `ScannedProduct`. Se il record è assente, lancia una richiesta HTTP secondaria verso l'infrastruttura esterna di `UPCitemdb`. Ricevuta la risposta dal backend, il frontend mappa i valori restituiti (nome, brand e immagine) all'interno dell'oggetto `scannedProduct` e forza l'avanzamento del wizard allo stato `found`. In caso di mancato riscontro da entrambe le fonti, il sistema emette un alert visivo per informare l'utente.

Geolocalizzazione e mappatura dei punti vendita

- Funzione/i di riferimento: `loadNearbyStores()`, `filterStores()` (nel componente `camera.page.ts`).
- Logica e Flusso Dati: Dopo che l'utente inserisce manualmente il prezzo del prodotto trovato inizia lo step conclusivo. La funzione `loadNearbyStores()` interpella l'API HTML5 di geolocalizzazione nativa del browser per estrarre la latitudine e longitudine attuali del dispositivo. Con questi dati, l'algoritmo assembla una query specializzata e genera una chiamata asincrona all'Overpass API di OpenStreetMap, richiedendo l'elenco dei punti vendita pertinenti (elettronica, computer, telefonia, supermercati) confinati in un raggio geometrico di 10 chilometri. Il payload di risposta viene processato iterativamente e proiettato negli array locali `nearbyStores` e `filteredStores`. L'utente può ulteriormente scremare la lista client-side digitando testo nella barra di ricerca, attivando l'algoritmo di inclusione della funzione `filterStores()`.

Salvataggio dati e logica di Crowdsourcing

- Funzione/i di riferimento: `selectStoreAndSave()` (nel componente `camera.page.ts`), `saveScannedProduct()` (nel controller `scanController.js`).
- Logica e Flusso Dati: Selezionato il negozio geografico, il frontend finalizza la composizione dell'oggetto prodotto, allega il token di autenticazione agli header e invia una richiesta POST al backend per la persistenza. Il controller ricevente esegue un'operazione di *upsert* (aggiornamento se esistente, inserimento se nuovo) sul database, vincolando l'entità al barcode, al negozio specifico e all'ID dell'utente. Contestualmente, si avvia il motore di crowdsourcing: il backend interroga il database cercando altri utenti che abbiano già salvato il medesimo barcode, ma con un valore di `currentPrice` strettamente maggiore di quello appena rilevato. Qualora l'array degli

utenti interessati non risulti vuoto, il sistema genera un inserimento massivo di notifiche (`Notice.insertMany()`), avvisando in tempo reale i membri della community della nuova occasione economica disponibile in zona. Infine, il server risponde con successo e il frontend azzerà le variabili, riportando il wizard allo stato iniziale.

2.5.3 API esterne e dettagli tecnici

Integrazione Hardware tramite Capacitor

Per la cattura ottica, l'applicazione fa affidamento sul modulo `@capacitor/barcode-scanner`. Questo plugin agisce da strato intermedio (bridge) tra l'ambiente web di Ionic/Angular e i sistemi operativi iOS o Android, garantendo l'accesso ai permessi della fotocamera e gestendo nativamente il riconoscimento ottico dei pattern EAN/UPC.

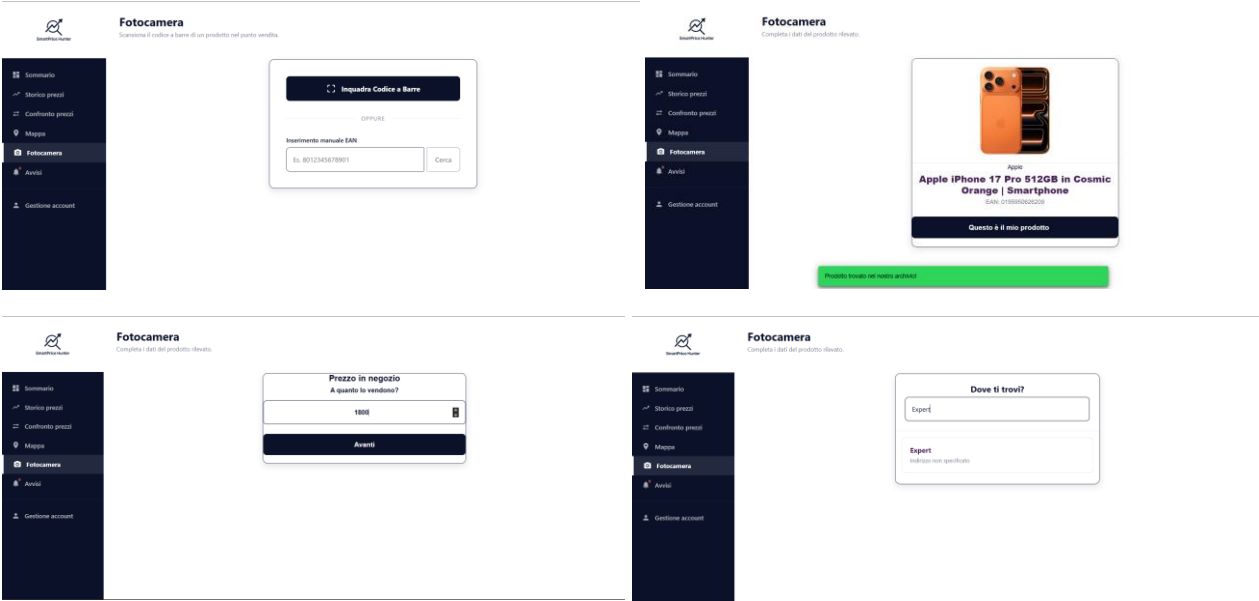
Motore spaziale Overpass API (OpenStreetMap)

Al fine di non appesantire l'infrastruttura con un database geografico proprietario dei negozi, l'algoritmo di ricerca geolocalizzata implementa chiamate dirette all'Overpass API. La stringa di interrogazione adotta il linguaggio dichiarativo Overpass QL, ordinando ai server di OpenStreetMap di scansionare nodi poligonali (`around:10000,lat,lon`) dotati di specifici tag semantici (`shop="electronics"`).

Risoluzione codici via UPCitemdb

Per massimizzare il tasso di riconoscimento dei codici a barre ignoti al database locale, l'architettura backend invia chiamate verso <https://www.upcitemdb.com/>. Questo servizio globale fornisce metadati normalizzati derivati dai grandi database della distribuzione commerciale mondiale, permettendo all'applicazione di autocompilare la scheda tecnica frontale senza richiedere l'inserimento manuale dei dati da parte dell'utente.

2.5.4 Mockup Pagina di Fotocamera



2.6 Pagina di Comparazione prezzi

La pagina di Comparazione prezzi offre all'utente una panoramica dei prodotti precedentemente scansionati fisicamente nei negozi. Oltre a visualizzare lo storico delle proprie rilevazioni locali, l'interfaccia permette di lanciare interrogazioni mirate per confrontare in tempo reale il prezzo fisico con le migliori offerte disponibili sugli e-commerce online.

2.6.1 Variabili di stato e Attributi

Le principali variabili di stato e gli attributi definiti all'interno della classe `PriceComparisonPage` includono:

- `localProducts`: Array delegato a contenere i prodotti locali pre-elaborati e raggruppati cronologicamente.
- `filteredProducts`: Array dinamico utilizzato per il rendering visivo sulla griglia, che subisce mutazioni in base alle ricerche e agli ordinamenti dell'ut
- `isLoadingLocal`: Flag booleano adibito a gestire il feedback visivo (buffering) durante la fase iniziale di recupero dati dal database.
- `sarchQuery`: Stringa delegata a memorizzare l'input testuale per il filtro di ricerca interno all'elenco.

2.6.2 Funzionalità e flusso logico

Caricamento e raggruppamento prodotti locali

- Funzione/i di riferimento: `loadLocalProducts()` (nel componente `price-comparison.page.ts`), `getLocalProducts()` (nel servizio `comparison.service.ts`), `getScannedProducts()` (nel controller `scanController.js`).
- Logica e Flusso Dati: All'avvio, il componente invoca il servizio per richiedere al backend l'elenco di tutte le scansioni fisiche effettuate dall'utente. Una volta ricevuto il payload, l'algoritmo front-end istanzia un oggetto `Map` per raggruppare dinamicamente i prodotti aventi il medesimo nome o codice a barre. In caso di corrispondenze multiple (stesso prodotto scansionato in punti vendita differenti), le posizioni vengono aggregate in un array interno `locations`. Successivamente, la mappa viene riconvertita in array e le rilevazioni vengono ordinate cronologicamente, impostando come riga attiva quella scansionata più di recente.

Gestione dinamica delle posizioni (Tendina Negozi)

- Funzione/i di riferimento: `toggleDropdown()`, `selectLocation()` (nel componente `price-comparison.page.ts`).

- Logica e Flusso Dati: Se un prodotto vanta più posizioni fisiche di scansione, l'utente può esplorarle tramite un menu a tendina. Il metodo `toggleDropdown()` gestisce lo stato di apertura assicurandosi di chiudere preventivamente eventuali altre tendine attive nella lista. Al click su un negozio specifico, `selectLocation()` aggiorna in tempo reale gli attributi del prodotto genitore (`_id`, `store`, e `currentPrice`) con i dati della selezione, chiudendo il menu e aggiornando istantaneamente la vista.

Ricerca online dei competitor

- Funzione/i di riferimento: `searchOnline()` (nel componente `price-comparison.page.ts`), `getOnlineCompetitors()` (nel servizio `comparison.service.ts`), `getOnlineCompetitors()` (nel controller `comparisonController.js`).
- Logica e Flusso Dati: L'utente innesca questa funzione per cercare i prezzi online di una specifica entità. Il sistema abilita un flag di caricamento dedicato (`isSearchingOnline`) e preleva il nome o il barcode del prodotto, delegando la chiamata HTTP al servizio. Il controller backend riceve la query e interroga l'infrastruttura esterna. Dopo aver validato la risposta, estrae e mappa i primi 5 risultati utili (`competitor`), ordinandoli esplicitamente in ordine di prezzo crescente (dal più economico). Il payload ottimizzato ritorna al frontend, che popola l'array `onlineCompetitors` del prodotto e rimuove lo stato di caricamento.

Eliminazione sicura del prodotto

- Funzione/i di riferimento: `deleteProduct()` (nel componente `price-comparison.page.ts`), `deleteLocalProduct()` (nel servizio `comparison.service.ts`), `deleteScannedProduct()` (nel controller `scanController.js`).
- Logica e Flusso Dati: Al tentativo di rimozione di una rilevazione, previa conferma visiva da parte dell'utente, viene inviata una richiesta DELETE all'endpoint associato. Il backend processa l'operazione imponendo un vincolo di sicurezza: l'eliminazione avviene solo se l'ID del prodotto e l'ID dell'utente loggato coincidono perfettamente, prevenendo manomissioni. Qualora la risposta del server sia positiva, l'algoritmo frontend elimina la specifica posizione dall'array `locations`. Se tale array si svuota, l'intero prodotto sparisce dalla griglia visibile; in caso contrario, il sistema si adatta impostando automaticamente in vista la rilevazione fisica immediatamente precedente.

Ordinamento e ricerca locale

- Funzione/i di riferimento: `presentSortOptions()`, `sortProducts()`, `filterProducts()`, `clearSearch()` (nel componente `price-comparison.page.ts`).
- Logica e Flusso Dati: La barra di ricerca innesca `filterProducts()`, un algoritmo case-insensitive che verifica l'inclusione della stringa digitata nel nome o nel barcode, proiettando i soli risultati nell'array `filteredProducts`. Per l'ordinamento, l'infrastruttura Ionic genera un menu nativo (Action Sheet) che offre scelte cronologiche o economiche. Il metodo `sortProducts()` riorganizza l'array visibile operando confronti matematici sui valori di `currentPrice` o, nel caso di "default", ripristinando la clonazione diretta dell'array originario.

2.6.3 API esterne e dettagli tecnici

Integrazione SerpApi (Google Shopping)

Il modulo di esplorazione dei prezzi online si appoggia a SerpApi per eseguire un web scraping strutturato ed estrarre i dati dal motore di Google Shopping in tempo reale. Per garantire massima precisione geografica e valutaria, le chiamate HTTP costruite nel controller backend prevedono i parametri di localizzazione e lingua italiani (`hl=it` e `gl=it`). L'algoritmo server-side implementa inoltre un pattern di ottimizzazione del payload: la mole di dati non elaborata viene analizzata e decurtata (`.slice(0, 5)`), permettendo la trasmissione client-side dei soli 5 competitor più rilevanti in un formato alleggerito e strutturato.

2.6.4 Mockup Pagina di Comparazione prezzi

Confronto prezzi
Qui puoi confrontare il prezzo dei tuoi prodotti su canali con i prezzi online

Prodotti rilevati in negozio

Prodotto Rilevato	Prezzo in Negozio (Fisico)	Migliori Offerte Online	Azioni
Apple iPhone 17 Pro 512GB in Cosmic ... EAN: 019095062029	€1800.00 Expert -	Cerca Prezzi Online	
Apple MacBook Neo (A18 Pro, 2024) L... EAN: 019095062029	€599.00 PC Elettronica Service	Cerca Prezzi Online	
Apple MacBook Neo (A18 Pro, 2024) L... EAN: 019095062029	€550.00 Euronics	Cerca Prezzi Online	
iMac 24" with Retina 4.5K display A18 L... EAN: 019095062029	€1300.00 Euronics -	Cerca Prezzi Online	
Apple iPhone Air 256GB in Cloud Whit... EAN: 019095062029	€899.99 ...	Cerca Prezzi Online	

Prodotti rilevati in negozio

Prodotto Rilevato	Prezzo in Negozio (Fisico)	Migliori Offerte Online	Azioni
Apple iPhone 17 Pro 512GB in Cosmic ... EAN: 019095062029	€1800.00 Expert -	Back Market 1221,00 € Riconfermate Vedi Offerta Futuroreus000 1470,00 € Neues Vedi Offerta Netfix S.A.S 1579,40 € Riconfermate Vedi Offerta Apple 1589,00 € Neues Vedi Offerta Apple 1739,00 € Neues Vedi Offerta	

2.7 Pagina di Mappa

La pagina di Mappa offre all'utente uno strumento cartografico interattivo avanzato. Essa permette di esplorare l'area circostante per individuare fisicamente i punti vendita di elettronica, e di localizzare su mappa le migliori offerte geolocalizzate dalla community per un prodotto specifico cercato, aiutando l'utente a trovare il prezzo più conveniente nelle proprie vicinanze.

2.7.1 Variabili di stato e Attributi

Le principali variabili di stato e gli attributi definiti all'interno della classe `MapPage` includono:

- `map` e `userMarker`: Referenze in memoria allocate all'istanza principale della mappa Leaflet e al marcatore circolare della posizione attuale dell'utente.
- `storeMarkers`: Array dinamico delegato a memorizzare e raggruppare i marcatori (pin) dei negozi o dei prodotti visualizzati, consentendo una facile pulizia del layer grafico.
- `searchQuery` e `searchResults`: Stringa di input dell'utente e array delegato a contenere i risultati univoci della ricerca globale dei prodotti in tempo reale.
- `isSearching`, `isSearchingDb`: Flag booleani adibiti a gestire i feedback visivi (buffering) rispettivamente durante le chiamate all'Overpass API e al backend interno.
- `hasSearchedNearby`: Flag booleano che indica se l'utente ha attivato la visualizzazione territoriale dei negozi di elettronica limitrofi.

2.7.2 Funzionalità e flusso logico

Inizializzazione della mappa e geolocalizzazione

- Funzione/i di riferimento: `initMap()`, `locateUser()` (nel componente `map.page.ts`).
- Logica e Flusso Dati: A valle della completa inizializzazione della vista (`ngAfterViewInit`), il metodo `initMap()` istanzia l'oggetto mappa di Leaflet centrandolo a livello nazionale, e vi applica il layer vettoriale standard di OpenStreetMap. Successivamente, tramite un ritardo asincrono differito per stabilizzare il DOM, invoca `locateUser()`. Questa funzione interpella le API di geolocalizzazione nativa HTML5 per acquisire le coordinate del dispositivo. Ottenuti i dati, l'algoritmo innesca un'animazione fluida della telecamera (`flyTo`) verso la posizione reale, ripulisce eventuali marker precedenti e disegna un marcatore circolare rosso (`L.circleMarker`) accompagnato da un popup identificativo dell'utente.

Ricerca negozi limitrofi tramite Overpass API

- Funzione/i di riferimento: `findNearbyElectronics()`, `clearStoreMarkers()` (nel componente `map.page.ts`).
- Logica e Flusso Dati: Attivata dal pannello utente, la funzione estrae la posizione GPS attuale e attiva lo stato di caricamento. L'algoritmo assembla dinamicamente una query in linguaggio Overpass QL volta a rintracciare nodi, percorsi e relazioni con il tag semantico `shop="electronics"`, circoscritti in un raggio geometrico di 10 chilometri. Il frontend esegue un fetch HTTP verso l'endpoint esterno. Ricevuta e destrutturata la risposta JSON, il sistema purifica la mappa chiamando `clearStoreMarkers()` e itera sugli elementi, estrapolando coordinate e nomi dei punti vendita. Istanziando nuovi marcatori sulla mappa e spingendoli nell'array `storeMarkers`, la funzione conclude istruendo la libreria grafica a ricalcolare e ottimizzare il livello di zoom per inquadrare perfettamente l'intero raggruppamento (`fitBounds`).

Ricerca globale dei prodotti in tempo reale

- Funzione/i di riferimento: `onSearchInput()` (nel componente `map.page.ts`), `searchProducts()` (nel controller `scanController.js`).
- Logica e Flusso Dati: Digitando un minimo di due caratteri, si innesca una richiesta GET asincrona al server backend. Il controller cattura la query e costruisce un'espressione regolare (Regex) case-insensitive. Ignorando deliberatamente i vincoli di proprietà utente, esegue una scansione globale del database `ScannedProduct`, limitando la risposta ai primi 10 documenti che matchano per nome o codice a barre. Tornato al client, il payload subisce un filtraggio locale per estirpare i duplicati di prodotto (basato sull'univocità del barcode), sovrascrivendo l'array visivo `searchResults` adibito al rendering della tendina.

Localizzazione e proiezione spaziale del prodotto

- Funzione/i di riferimento: `selectProduct()` (nel componente `map.page.ts`), `getProductLocations()` (nel controller `scanController.js`).
- Logica e Flusso Dati: Alla selezione di un prodotto, il sistema richiede le coordinate correnti dell'utente per stabilire un epicentro. Invia poi il codice a barre all'endpoint pubblico del backend. Il controller interroga il DB ed estrae l'intera storicità di quel prodotto, ordinando i record in modo decrescente. Sfruttando un costrutto Set come matrice di filtraggio per la chiave "nome-indirizzo", l'algoritmo backend elimina le registrazioni multiple per un medesimo negozio fisico, trasmettendo al client un array compatto contenente solo l'ultima rilevazione di prezzo per ciascun punto vendita unico. Nel client, un'ulteriore logica spaziale calcola matematicamente la

distanza lineare (`distanceTo`) tra l'utente e il punto vendita. Se la distanza rientra nel raggio di 10 km, il sistema formatta il prezzo e genera un indicatore interattivo sulla mappa, altrimenti scarta il dato notificando all'utente l'assenza di offerte locali rilevanti.

2.7.3 API esterne e dettagli tecnici

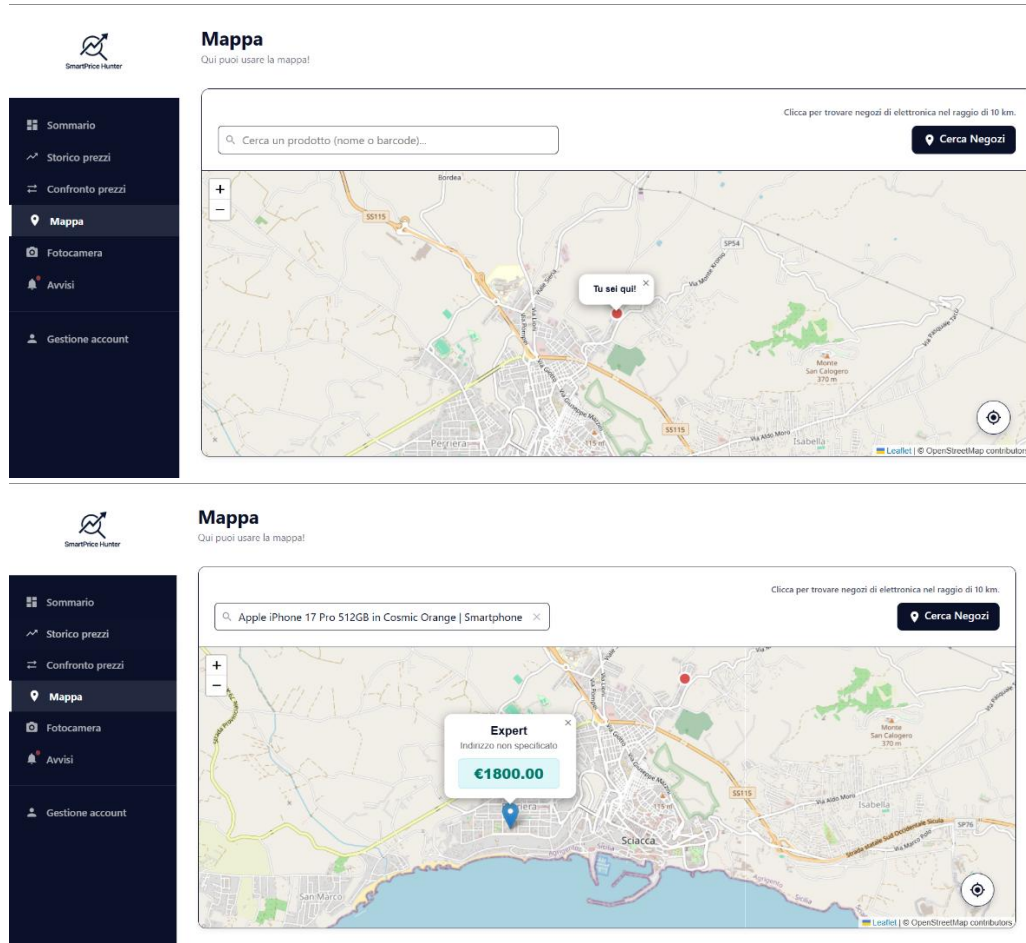
Integrazione Cartografica Leaflet.js

Per il rendering vettoriale e la complessa interazione spaziale, l'applicativo fa perno sulla libreria open-source Leaflet.

Integrazione Overpass API e Overpass QL

La mappatura territoriale dei negozi viene esternalizzata all'Overpass API. La chiamata stringa adotta il linguaggio di query Overpass QL (es. `node["shop"]="electronics"(around:10000,lat,lon)`), che comanda al server di isolare ed estrarre porzioni precise di dati OpenStreetMap. Questa soluzione architetturale favorisce scalabilità e manutenibilità, fornendo dati sempre in tempo reale basati sulla geolocalizzazione istantanea.

2.7.4 Mockup Pagina di Mappa



2.8 Pagina di Avvisi

La pagina degli Avvisi (o Notifiche) centralizza le comunicazioni di sistema e le segnalazioni della community relative alle occasioni sui prodotti (Crowdsourcing). Consente all'utente di visionare, filtrare per categoria, contrassegnare come lette e cestinare le notifiche ricevute.

2.8.1 Variabili di stato e Attributi

Le principali variabili di stato e gli attributi definiti all'interno della classe `NoticesPage` includono:

- `notices`: Array dinamico delegato a contenere la lista completa degli avvisi scaricati dal database.
- `activeFilter`: Stringa di stato (con valori predefiniti 'all', 'unread', 'prices', 'system') che governa il criterio di visualizzazione corrente nella vista utente.
- `unreadCount` e `unreadSub`: Variabile numerica e relativa sottoscrizione (Subscription) reattiva, adibite a memorizzare e mantenere aggiornato in tempo reale il contatore globale ("bollino") delle notifiche ancora da leggere.

2.8.2 Funzionalità e flusso logico

Inizializzazione e gestione reattiva dello stato

- Funzioni di riferimento: `ngOnInit()`, `ngOnDestroy()`, `loadNotices()` (nel componente `notices.page.ts`), `getNotices()` (nel servizio `notices.service.ts`), `getNotices()` (nel controller `noticeController.js`).
- Logica e Flusso Dati: All'apertura della pagina, `ngOnInit()` innesca due flussi operativi paralleli. Da una parte, si iscrive al flusso reattivo `unreadCount$` (BehaviorSubject) esposto dal servizio per mantenere sincronizzato il contatore visuale, garantendo di annullare la sottoscrizione alla distruzione del componente tramite `ngOnDestroy()` per evitare perdite di memoria. Dall'altra, invoca `loadNotices()` che esegue una richiesta HTTP GET. Il controller intercetta la richiesta, verifica il token dell'utente e interroga il database restituendo tutti gli avvisi associati, ordinandoli cronologicamente dal più recente al più vecchio. Alla ricezione del payload nel servizio, il sistema sfrutta l'operatore `RxJS tap()` per contare contestualmente gli elementi con `read: false`, trasmettendo il nuovo totale all'intera applicazione prima di passare i dati al componente per il rendering.

Filtraggio dinamico degli avvisi

- Funzioni di riferimento: `filteredNotices`, `setFilter()` (nel componente `notices.page.ts`).

- Logica e Flusso Dati: L'utente può scremare la mole di messaggi interagendo con i controlli della UI, che invocano il metodo `setFilter()` per alterare il valore di `activeFilter`. Il template HTML è ancorato al getter `filteredNotices`, che agisce come una proprietà calcolata ad alte prestazioni: iterando sull'array originale `notices`, applica uno statement condizionale e proietta nella vista esclusivamente gli elementi che soddisfano i criteri logici della categoria selezionata (es. `!n.read` per i non letti, oppure `n.type === 'price_drop'` per le offerte scoperte dalla community).

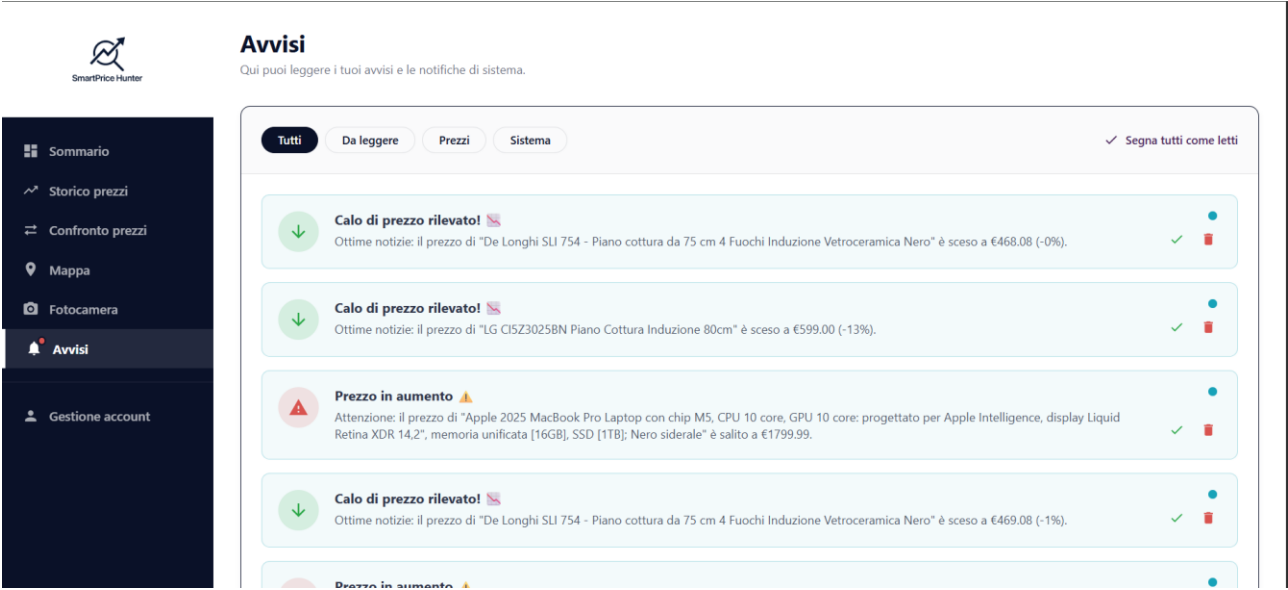
Letture singola e globale degli avvisi

- Funzioni di riferimento: `markAsRead()`, `markAllAsRead()` (nel componente `notices.page.ts`), `markAsRead()`, `markAllAsRead()` (nel controller `noticeController.js`).
- Logica e Flusso Dati: L'interazione con una specifica notifica o col pulsante globale innesca una chiamata HTTP PUT asincrona verso il backend. Il server localizza il documento (o i documenti multipli tramite `updateMany`) appartenente all'utente loggato e aggiorna il database impostando il flag booleano `read` a vero. Per massimizzare le prestazioni ed evitare costosi refresh di rete, l'algoritmo client-side adotta una strategia di aggiornamento "ottimistico": muta localmente lo stato `read` dell'avviso nell'array `notices` ed esegue un ricalcolo matematico dei messaggi non letti rimasti, iniettando il risultato nel metodo `updateUnreadCount()` per sincronizzare istantaneamente il badge globale. In caso di lettura globale, il servizio azzerava direttamente il contatore propagando uno 0 nel canale reattivo.

Eliminazione di un avviso

- Funzioni di riferimento: `deleteNotice()` (nel componente `notices.page.ts`), `deleteNotice()` (nel controller `noticeController.js`).
- Logica e Flusso Dati: La rimozione di un messaggio attiva una richiesta HTTP DELETE mirata al server. Il controller esegue la cancellazione del nodo dal database (`findOneAndDelete`), garantendo un controllo di sicurezza incrociato sull'ID dell'avviso e sull'ID dell'utente. Ottenuta la conferma dal server, la funzione frontend purifica l'array visivo eliminando l'elemento rimosso e proietta un feedback visivo (Toast) all'utente. L'algoritmo conclude ricalcolando ed emettendo il nuovo ammontare delle notifiche non lette, poiché l'eliminazione diretta di un avviso ancora da visionare comporta necessariamente il decremento del bollino informativo.

2.8.3 Mockup Pagina di Avvisi



2.9 Pagina di Gestione account

La pagina di Gestione account consente all'utente di amministrare il proprio profilo personale in totale autonomia. Tramite questa interfaccia è possibile aggiornare lo username, modificare la password di accesso rispettando rigorosi criteri di sicurezza, effettuare il logout o richiedere l'eliminazione irreversibile dell'account e di tutti i dati ad esso associati.

2.9.1 Variabili di stato e Attributi

Le principali variabili di stato e gli attributi definiti all'interno della classe `ManageAccountPage` includono:

- `usernameForm` e `passwordForm`: Oggetti di tipo `FormGroup` delegati alla gestione reattiva, alla validazione e alla raccolta dei dati immessi nei rispettivi moduli di aggiornamento.
- `showCurrentPassword`, `showNewPassword`, `showConfirmPassword`: Flag booleani che governano l'interazione dell'utente con le icone a forma d'occhio, alterando lo stato di visibilità in chiaro dei campi password.
- `passwordStrength`: Variabile intera (da 0 a 3) che quantifica dinamicamente la robustezza della nuova password durante la digitazione, utile per fornire un feedback visivo immediato tramite una barra colorata.
- `newPasswordError`: Stringa delegata a memorizzare e mostrare specifici messaggi di errore in tempo reale qualora la nuova password non soddisfi strutturalmente i requisiti minimi di sicurezza.

2.9.2 Funzionalità e flusso logico

Valutazione in tempo reale della sicurezza della password

- Funzione/i di riferimento: `onPasswordInput()`, `passwordMatchValidator()` (nel componente `manage-account.page.ts`).
- Logica e Flusso Dati: Il metodo `onPasswordInput()` viene innescato ad ogni singola digitazione dell'utente all'interno del campo della nuova password. L'algoritmo valuta progressivamente la stringa allocando punti alla variabile `passwordStrength` in base alla lunghezza complessiva e alla presenza di maiuscole, numeri e caratteri speciali. Parallelamente, identifica l'esatto criterio mancante e popola la stringa `newPasswordError` con istruzioni testuali mirate (es. "Aggiungi almeno una lettera maiuscola."). Infine, la logica di `passwordMatchValidator()` agisce a livello di form, verificando la perfetta congruenza logica tra la nuova password e il campo di conferma prima di sbloccare l'operazione di salvataggio.

Aggiornamento dello username

- Funzione/i di riferimento: `updateUsername()` (nel componente `manage-account.page.ts`), `updateUsername()` (nel controller `accountController.js`).
- Logica e Flusso Dati: Previa validazione locale del form, il metodo frontend impacchetta il nuovo username e lo trasmette tramite una richiesta HTTP PUT al backend, allegando il token JWT di autenticazione all'interno degli header. Il controller ricevente verifica la consistenza dell'input ed esegue un controllo di unicità sul database interrogando il modello `User`. Se lo username risulta già assegnato, il server blocca l'operazione restituendo un errore specifico per segnalare la violazione; in caso contrario, aggiorna il documento dell'utente restituendo un payload privo di dati sensibili. Alla ricezione del successo dal server, l'interfaccia client-side si resetta e notifica l'esito all'utente tramite `Toast`.

Modifica della password

- Funzione/i di riferimento: `changePassword()` (nel componente `manage-account.page.ts`), `changePassword()` (nel controller `accountController.js`).
- Logica e Flusso Dati: Inibita in caso di form invalido, questa funzione innesca una richiesta HTTP PUT contenente la password attuale e la nuova. Il controller estrapola l'identificativo utente e sfrutta la libreria `bcrypt` per comparare crittograficamente la password attuale fornita con l'hash depositato nel database. Una volta accertata l'identità, il backend riapplica ferrei controlli (Regex) sulla nuova stringa per garantirne l'elevata complessità strutturale. Superati i controlli, genera un nuovo "salt" crittografico, esegue l'hashing della nuova password e sovrascrive il record. Il frontend reagisce pulendo la vista, svuotando i campi e azzerando gli indicatori visivi di forza.

Eliminazione sicura dell'account e dei dati associati

- Funzione/i di riferimento: `confirmDeleteAccount()`, `deleteAccount()` (nel componente `manage-account.page.ts`), `deleteAccount()` (nel controller `accountController.js`).
- Logica e Flusso Dati: L'intenzione di eliminare l'account genera un alert nativo che richiede imperativamente l'inserimento della password attuale, fungendo da estrema barriera di sicurezza. Alla conferma, `deleteAccount()` assembla una richiesta HTTP DELETE iniettando la stringa della password nel corpo della chiamata. Il controller backend verifica l'esattezza della password decifrata tramite `bcrypt`. In caso di corrispondenza positiva, il sistema esegue un'operazione architetturale di "Cascade Delete": dapprima interroga il database per rimuovere massivamente (`deleteMany`) tutti i record di tracciamento prodotti orfani associati all'ID dell'utente,

preservando così l'integrità relazionale, e solo in seguito elimina fisicamente il documento dell'utente stesso. Il frontend, accertato l'esito, purifica la memoria locale (`localStorage.clear()`) e instrada il router verso la pagina di login.

Disconnessione (Logout)

- Funzione/i di riferimento: `logout()` (nel componente `manage-account.page.ts`).
- Logica e Flusso Dati: Attivata dal pulsante di uscita, questa funzione esegue una pulizia totale rimuovendo le variabili di sessione, il token di sicurezza e i metadati dell'utente allocati all'interno della `localStorage` della memoria del browser. Successivamente, genera un messaggio visivo di conferma e invoca il router per forzare la navigazione verso la schermata pubblica (`/login`), inibendo le rotte protette.

2.9.3 Mockup Pagina di Gestione account



Gestione account

Qui puoi gestire i tuoi dati personali!

Sommario

Storico prezzi

Confronto prezzi

Mappa

Fotocamera

Avvisi

Gestione account

Dati profilo

Aggiorna le informazioni pubbliche associate al tuo profilo.

Nuovo username

Es. MarioRossi99

Aggiorna username

Sicurezza account

Modifica regolarmente la tua password per mantenere sicuro il tuo account.

Password attuale

Inserisci la password attuale

Nuova password

Crea nuova password

Conferma nuova password

Ripeti la nuova password

Aggiorna password



Sommario

Storico prezzi

Confronto prezzi

Mappa

Fotocamera

Avvisi

Gestione account

Password attuale

Inserisci la password attuale

Nuova password

Crea nuova password

Conferma nuova password

Ripeti la nuova password

Aggiorna password

Disconnessione

Termina la sessione corrente in modo sicuro.

Esci dall'account

Elimina account

Le operazioni in quest'area cancellano in modo permanente i tuoi dati e non sono reversibili. Eliminando l'account, tutti i tuoi prodotti monitorati e lo storico dei prezzi salvato andranno persi per sempre.

Elimina account permanentemente

2.10 Pannello Admin (raggiungibile solo attraverso l'url <http://localhost:8100/admin-users>)

Il Pannello Admin rappresenta il pannello di controllo riservato agli utenti amministratori di sistema. Tramite questa interfaccia privilegiata, l'amministratore può monitorare l'intera base di iscritti all'applicazione, effettuare ricerche mirate, interdire o ripristinare l'accesso alla piattaforma per specifici profili (Ban/Sban) ed eliminare permanentemente gli account dal database.

La creazione di un utente admin è delegata a un apposito script "`..\backend\createAdmin.js`", basta runnare questo script e verrà creato un admin con email : admin@smartprice.com e password : Admin123!

2.10.1 Variabili di stato e Attributi

Le principali variabili di stato e gli attributi definiti all'interno della classe AdminUsersPage includono:

- `users`: Array dinamico che agisce come fonte di verità, delegato a contenere i dati grezzi di tutti gli utenti scaricati dal database.
- `filteredUsers`: Array dinamico utilizzato per il rendering visivo sulla griglia, che subisce mutazioni continue in base alle interrogazioni del motore di ricerca interno.
- `searchQuery`: Stringa delegata a memorizzare l'input testuale immesso dall'amministratore nella barra di ricerca.
- `isLoading`: Flag booleano adibito a governare il feedback visivo (buffering) durante le latenze di rete per le chiamate al server.
- `isSidebarActive`: Flag booleano preposto alla gestione della visibilità del menu laterale in contesti di visualizzazione mobile.

2.10.2 Funzionalità e flusso logico

Protezione della rotta e Verifica dei privilegi

- Funzione/i di riferimento: `canActivate()` (nella guardia `admin.guards.ts`).
- Logica e Flusso Dati: Prima di consentire il caricamento del componente visivo, l'architettura di routing (file `app.routes.ts`) invoca una guardia dedicata. Questo metodo ispeziona la `localStorage` del browser per recuperare l'oggetto `user` della sessione corrente. Analizza quindi l'attributo `role`: se l'utente possiede il ruolo esatto di `'admin'`, la funzione restituisce `vero` autorizzando la navigazione. In caso contrario (utente standard o non autenticato), l'algoritmo blocca la transizione, genera un messaggio visivo di diniego ("Accesso negato. Area riservata...") e instrada forzatamente

l'utente verso la pagina di login, blindando la rotta da accessi non autorizzati.

Inizializzazione e recupero degli iscritti

- Funzione/i di riferimento: `ngOnInit()`, `loadUsers()` (nel componente `admin-users.page.ts`), `getAllUsers()` (nel servizio `admin.service.ts`), `getAllUsers()` (nel controller `adminController.js`).
- Logica e Flusso Dati: Il ciclo di vita del componente inizia con `ngOnInit()` che innesca `loadUsers()`. La funzione abilita il flag `isLoading` a vero e delega al servizio l'esecuzione di una richiesta HTTP GET verso l'endpoint `/api/admin/users`, iniettando il token JWT negli header di autorizzazione. Il controller backend prende in carico la richiesta e interroga il database MongoDB (`User.find()`). Per rigorose policy di sicurezza, l'algoritmo server-side applica una proiezione (`.select('-password')`) al fine di escludere le credenziali sensibili dal payload di risposta. Alla ricezione dei dati sul client, il sistema popola l'array `users` e ne effettua una clonazione in `filteredUsers` per l'immediato rendering in tabella, spegnendo infine il flag di caricamento.

Motore di ricerca client-side degli utenti

- Funzione/i di riferimento: `filterUsers()` (nel componente `admin-users.page.ts`).
- Logica e Flusso Dati: L'amministratore può filtrare la lista interagendo con l'apposita barra. A ogni input, la funzione acquisisce la stringa `searchQuery`, eliminando gli spazi vuoti laterali (`.trim()`) e forzando i caratteri in minuscolo (`.toLowerCase()`). L'algoritmo esegue un'iterazione sull'array primario `users`, valutando se la sottostringa di ricerca sia inclusa all'interno dei campi `email` o `username` (anch'essi normalizzati in minuscolo). I record che soddisfano la condizione vengono proiettati nell'array `filteredUsers`. Qualora il campo di ricerca venga svuotato, il sistema ripristina la totalità dei dati clonando nuovamente l'intero array originale.

Gestione delle restrizioni di accesso (Ban / Unban)

- Funzione/i di riferimento: `toggleBanStatus()` (nel componente `admin-users.page.ts`), `banUser()`, `unbanUser()` (nel servizio `admin.service.ts` e nel controller `adminController.js`).
- Logica e Flusso Dati: L'interazione con il pulsante di blocco/sblocco intercetta lo stato attuale dell'utente target (`isBanned`) e genera una finestra nativa (`Alert`) per richiedere conferma esplicita dell'azione all'amministratore. Alla conferma, il flusso si dirama: se l'utente è attualmente bannato, il servizio invia una chiamata HTTP POST all'endpoint di `sban (/unban)`; viceversa, all'endpoint di `ban (/ban)`. In entrambi gli scenari, il

controller backend individua l'utente tramite l'ID fornito (findByldAndUpdate) e inverte il valore booleano sul database. Ricevuto l'esito positivo dal server, il frontend emette un messaggio di notifica semantico (verde per riattivazione, giallo/warning per il ban) e invoca immediatamente loadUsers() per risincronizzare la tabella con i dati aggiornati.

Eliminazione irreversibile dell'utente

- Funzione/i di riferimento: deleteUser() (nel componente admin-users.page.ts), deleteUser() (nel servizio admin.service.ts e nel controller adminController.js).
- Logica e Flusso Dati: La richiesta di rimozione di un account attiva un Alert critico (Danger) che avverte l'amministratore dell'irreversibilità dell'operazione. Confermata l'intenzione, il servizio assembla una richiesta HTTP DELETE indirizzata al backend. Il controller esegue l'eliminazione fisica del documento dal database (findByldAndDelete) sfruttando l'ID univoco. Una volta che il server conferma la distruzione del record, il componente frontend notifica l'amministratore del successo dell'operazione e lancia un nuovo ciclo di caricamento (loadUsers()) per purificare la griglia visiva, rimuovendo definitivamente l'utente estinto dall'interfaccia.

2.10.3 Mockup Pannello Admin

Gestione Utenti

Pannello di amministrazione per il controllo degli iscritti.

Utente	Ruolo	Stato	Azioni
S superadmin admin@smartprice.com	Amministratore	Attivo	
D davide_ciccio davide@example.com	Utente	Attivo	